Welcome to

The Session Will Begin Shortly

START

Web Scraping and Data Collection

Day 1: From Web Source to Raw Dataset

Moses Boudourides

Faculty, Graduate Program on Data Science
Northwestern University

Moses.Boudourides@northwestern.edu
Moses.Boudourides@gmail.com

instats Seminar

Wednesday, May 13, 2026
4:00 PM – 5:30 PM UTC

Northwestern | SCHOOL OF PROFESSIONAL STUDIES

Day 1: From Web Source to Raw Dataset

1. Why Web-Based Data Collection Matters
2. Theoretical and Methodological Foundations
3. From Source to Dataset: Core Concepts
4. Extraction, Field Construction, and Bias
5. Applications: Health, Life, and Social Sciences
6. Technical Considerations and the Streamlit App
7. Practical Session, Outcomes, and Day 2 Preview

A No-Code Seminar Built on Transparent, Open Scripts

This seminar does **not** require participants to write or modify code. All operations are performed through a Streamlit application. However, every operation in the app corresponds to inspectable Python and R scripts hosted in a public GitHub repository. Participants learn the **logic** of digital data construction without being forced prematurely into programming.

- Target audience: early-career academics, advanced doctoral students
- Disciplines: Health Sciences, Life Sciences, Social Sciences
- Tools: Streamlit app (no-code interface) + Python/R repository (transparent backend)

Online Sources Have Become Indispensable to Empirical Research

Public institutions, registries, legislative bodies, laboratories, and funding portals increasingly publish structured information online. For researchers in the life and social sciences, the web is not merely a communication medium — it is a **field site, an archive, and a distributed infrastructure** for observable records (Boudourides 2025; Lazer et al. 2009).

The methodological consequence is direct: if research increasingly depends on online information, then the collection, organization, and evaluation of web-based data must be treated as a **serious part of research design**, not an informal preliminary task.

Web Scraping Is a Stage in Data Construction, Not a Standalone Trick

The central argument of this seminar is that web scraping is best understood as **one component of a broader research workflow**. That workflow begins with a substantive question, proceeds through source identification and unit definition, and continues through extraction, cleaning, documentation, validation, and interpretation.

> *"Web scraping is less a narrow programming trick than a practical and epistemic bridge between online information environments and research-ready datasets." — Boudourides (2025)*

Digital Environments Generate New Forms of Social Data at Scale

The emergence of computational social science established that digital environments create unprecedented opportunities for measurement and analysis (Lazer et al. 2009, 2020). However, the central challenge is not simply the availability of digital traces but the capacity to **align those traces with meaningful research questions**, ethically acceptable collection strategies, and defensible validation procedures (Grimmer et al. 2022; Ohme et al. 2024).

Web Scraping Borrows from Text-as-Data but Poses Distinct Problems

Grimmer, Roberts, and Stewart (2022) argue that data construction is inseparable from theory, representation, and validation. These insights are highly useful for web-derived data, but the relationship is **analogical rather than strictly equivalent**. In web scraping, the researcher must navigate DOM hierarchies, pagination schemes, hidden fields, and changing interface logic — challenges that text-as-data frameworks do not address directly.

Websites Present Already-Formatted, Institutionally Shaped Representations

Kitchin (2014) and van Dijck, Poell, and de Waal (2018) emphasize that digital records are shaped by socio-technical infrastructures and platform governance. A website does not merely host information — it presents information according to **page architectures, menus, templates, filters, visibility rules, and update practices** that affect what the scraper can observe.

Three Broad Source Categories Cover Health, Life, and Social Sciences

Domain	Source Types	Examples
Health Sciences	Registries, directories, policy portals	ClinicalTrials.gov, WHO GHO, hospital directories
Life Sciences	Lab pages, funding databases, species registries	NIH RePORTER, GBIF, institutional repositories
Social Sciences	Legislative portals, org websites, public records	GovInfo, municipal portals, advocacy directories

The key criterion is **repeated structure**: a source is suitable when its units can be aligned to consistent rows and variables in a dataset.

Online Information Becomes Research Data Through Principled Transformation

Three distinctions anchor the first hour of Day 1:

1. **Online information** — raw content on a webpage, not yet structured for analysis
2. **Extractable structure** — repeated patterns that can be mapped to rows and columns
3. **Research-ready data** — a validated, documented tabular dataset aligned to a research question

The movement from (1) to (3) is not automatic. It requires source selection, unit definition, field construction, cleaning, and validation — all of which are covered across the three days.

Question-Led Design Determines Which Sources Are Appropriate

A robust web-based data collection project begins with a **research question**, not an available website. Question-led design affects source selection in three ways:

- It determines what **population or domain** the source should represent
- It determines which **fields matter** (a source rich in text but lacking stable dates may be unsuitable for temporal comparison)
- It determines the acceptable level of **incompleteness or noise**

"Methods are useful only in relation to tasks, and tasks derive from substantive inquiry." — Grimmer et al. (2022)

Repeated Page Elements Are the Raw Material of Structured Extraction

A website is suitable for extraction when it contains **repeated structural units** — entries in a registry, rows in a directory, profiles in a listing. These units correspond to the rows of a future dataset. The researcher's task is to identify:

- What constitutes **one record** (the unit of analysis)
- Which **fields** are consistently present across records
- Whether the structure is **stable** enough to support systematic extraction

Unclear Unit Definition Produces Datasets Whose Rows Mean Different Things

One of the most consequential steps in web-based data construction is defining the **unit of analysis**. Web pages often contain nested structures: a page may represent an institution, while sub-elements represent programs, persons, or events.
Without explicit unit definition, a researcher can easily **mix levels of analysis** or produce a dataset whose rows do not correspond consistently to a meaningful entity. A clear statement of what one row in the final dataset represents is indispensable and must precede field extraction.

Scraping Is Not Only Extraction — It Is a Measurement Decision

Once the unit of analysis is defined, the researcher must identify the **fields** that will represent each unit. A provisional schema helps: title, identifier, date, institution, category, location, URL, and notes are generic fields that recur across domains.

Scraping introduces a **measurement layer** that mediates between the website and the dataset. CSS selectors, XPath expressions, table boundaries, and pagination rules all shape selective observability. What becomes visible in the final dataset is partly a function of source content and partly a function of how the extraction logic models that content.

Web-Based Data Inherit Multiple Forms of Bias That Must Be Acknowledged

Drawing on Grimmer et al. (2022), four forms of bias apply to web scraping:

Bias Type	**Description**
Resource bias	Some institutions maintain richer, more accessible pages
Incentive bias	Actors selectively disclose information for strategic reasons
Medium bias	Websites privilege some kinds of information and omit others
Retrieval bias	Collection captures only pages with certain access paths

A fifth form — **structural bias (DOM-dependent extraction bias)** — arises when page architecture systematically affects what is retrievable.

Clinical Trial Registries Offer Structured, Publicly Accessible Health Data

ClinicalTrials.gov is a registry of publicly and privately supported clinical studies. Its v2 API provides structured JSON records for each trial, including:

- NCT identifier, brief title, overall status
- Lead sponsor name, conditions studied, study type

Day 1 task: Fetch 50 trials related to *diabetes* via the API, inspect the raw JSON structure, and identify the unit of analysis (one trial = one row). The raw output reveals nested fields, inconsistent condition labels, and missing sponsor information — all of which become cleaning tasks on Day 2.

The WHO GHO API Provides Country-Level Health Indicators as Open Data

The WHO Global Health Observatory (GHO) exposes health indicators through the Athena OData API. Indicator 'WHOSIS_000001' (life expectancy at birth) returns records with:

- Country code and spatial dimension type
- Year, sex disaggregation, and numeric value

Day 1 task: Fetch 100 records for life expectancy, inspect the raw JSON, and note the presence of multiple sex categories per country-year combination — a structural feature that must be handled during cleaning on Day 2.

NIH RePORTER Exposes Funded Research Grants Through a Public API

The NIH Research Portfolio Online Reporting Tools (RePORTER) API allows keyword-based search across funded projects. A search for *genomics* returns records including:

- Project number, title, abstract
- Organization name (nested), agency, award amount

Day 1 task: Fetch 50 grants, inspect the raw JSON, and identify the nested structure of organization and agency fields. The raw output demonstrates how real API responses embed related entities within records rather than providing flat tables.

GovInfo Provides Structured Access to U.S. Federal Legislative Records

The GovInfo API (U.S. Government Publishing Office) exposes collections of federal documents including bills, regulations, and congressional records. The BILLS collection returns:

- Package ID, title, Congress number
- Date issued, document class (hr, s, etc.)

Day 1 task: Fetch 50 recent bills, inspect the raw JSON, and note the minimal field set — a reminder that legislative portals may be strong for official procedural records but weak for informal deliberation or legislative history.

Assessing a Source Means Asking What It Can and Cannot Represent

Source assessment involves four practical questions:

1. **What is the unit?** (trial, grant, bill, institution, species occurrence)
2. **Which fields are consistently present?** (identifier, date, category, location)
3. **What is missing or irregular?** (nested fields, inconsistent labels, missing values)
4. **What does the source not capture?** (informal activity, historical records, non-registered entities)

These questions transform extraction from a technical convenience into a **reproducible research decision** with a documented rationale.

Silent Partial Extraction Is the Most Consequential Risk in Web Scraping

Websites are not static objects. Their HTML structure may change due to redesigns or A/B testing. Key failure modes include:

Failure Mode	Cause	Consequence
Silent partial extraction	Incomplete pagination, conditional loading	Missing observations with no error message
Dynamic rendering	JavaScript-loaded content	Empty or incomplete HTML response
Rate limiting / CAPTCHA	Server-side access controls	Incomplete collection, selection bias
Schema drift	Source redesign	Broken selectors, incorrect field mapping

Recognizing these risks is essential for designing robust collection procedures and interpreting data quality.

Local Caching Ensures Reproducibility Even When Sources Change

All scripts in this seminar implement a **cache-first strategy**: before making any network request, the script checks whether a local copy of the data already exists. If it does, the cached file is loaded. If not, the data is fetched and saved.
This approach serves three purposes:

- **Reproducibility**: computations can be repeated without re-fetching
- **Resilience**: scripts work even when the source is temporarily unavailable
- **Transparency**: the raw data file is preserved alongside the cleaned version

Cache files are excluded from the GitHub repository via ‘.gitignore‘ but the folder structure is preserved with ‘.gitkeep‘ placeholders.

Both Python and R Scripts Follow the Same Logic with Domain-Appropriate Libraries

Task	Python Libraries	R Libraries
HTTP requests	‘requests‘	‘httr2‘
JSON parsing	‘json‘ (stdlib)	‘jsonlite‘
Data manipulation	‘pandas‘	‘dplyr‘, ‘purrr‘
File system	‘os‘, ‘pathlib‘	‘fs‘
Caching	file-based (CSV/JSON)	file-based (CSV/JSON)

Both implementations produce identical outputs. Participants may use whichever language they are more comfortable with, or compare the two to understand how the same operation is expressed differently.

The Streamlit App Exposes the Workflow Through Menus and Templates

The Streamlit application (Phase II of the seminar) provides a modular, no-code interface to the same operations performed in the scripts. Its five modules correspond directly to the research workflow:

Module	**Function**
1. Start / Choose input	Single URL, URL list, CSV/Excel, or existing dataset
2. Collect	Guided extraction using source templates
3. Clean	Duplicate removal, date standardization, label harmonization
4. Explore	Summary tables, missingness checks, simple plots
5. Export	Cleaned datasets, metadata sheets, processing summaries

The App Accepts Multiple Input Types to Serve a Mixed Academic Audience

Input Type	Use Case	App Behavior
Single URL	Test one webpage or profile page	Preview source, suggest templates
List of URLs	Many similar pages to collect	Process as repeated units, combine outputs
CSV/Excel with URLs	Target list with identifiers	Use file as control sheet, append extracted variables
CSV/Excel dataset (no URLs)	Already have data, want to clean	Skip extraction, open cleaning modules
Raw scraped output	Unfinished collection from another workflow	Focus on restructuring, validation, export

Participants Inspect Raw Output and Identify Its Limitations

In the third hour of Day 1, participants use the Streamlit app to:

1. Enter one or more URLs from the provided examples
2. Preview the source structure
3. Select a source template (listing, profile, directory, table)
4. Run the first guided collection workflow
5. Inspect the raw output and compare fields across records

The goal is not technical complexity but **conceptual understanding**: how does the app convert a website or list of URLs into a raw dataset, and what are the obvious limitations of that raw output?

Raw JSON from ClinicalTrials.gov Reveals Nested Structure and Inconsistencies

A single raw record from the ClinicalTrials.gov API contains:

- Deeply nested JSON (e.g., ‘protocolSection → identificationModule → nctId‘)
- Conditions listed as an array of strings with inconsistent terminology
- Status values in mixed case (‘RECRUITING‘, ‘Completed‘, ‘not yet recruiting‘)
- Missing sponsor information for some records

These features are not defects in the source — they reflect how the registry was designed. Recognizing them is the first step toward principled cleaning on Day 2.

By End of Day 1, Participants Can Identify, Assess, and Extract from a Source

By the end of Day 1, participants should be able to:

- Identify a plausible online source for structured collection in their own field
- Understand what kind of data can be extracted from it and what cannot
- Define the unit of analysis for a given source
- Follow the logic by which the app converts a website or list of URLs into a raw dataset
- Recognize the most common limitations of raw web-derived outputs

Day 2 Transforms Raw Output into a Usable, Documented Dataset

Day 2 picks up exactly where Day 1 ends — with raw, messy JSON or HTML-derived tables. The focus shifts to:

- Identifying and resolving common data quality problems (inconsistent labels, duplicates, missing values, irregular dates)
- Applying no-code cleaning operations through the Streamlit app
- Comparing raw and cleaned versions of the same data
- Generating a processing log and metadata sheet for the output

References

- Boudourides, M. (2025). Web Scraping and Data Collection for Life and Social Sciences.
- Grimmer, J., Roberts, M. E., & Stewart, B. M. (2022). *Text as Data*. Princeton University Press.
- Kitchin, R. (2014). *The Data Revolution*. SAGE.
- Lazer, D. et al. (2009). Computational social science. *Science*, 323(5915), 721–723.
- Lazer, D. et al. (2020). Computational social science: Obstacles and opportunities. *Science*, 369(6507), 1060–1062.
- Ohme, J. et al. (2024). Digital trace data collection for social media effects research. *Communication Methods and Measures*, 18(2), 124–141.
- van Dijck, J., Poell, T., & de Waal, M. (2018). *The Platform Society*. Oxford University Press.

Questions and Discussion

Thank you!

Questions?

Moses.Boudourides@northwestern.edu
Moses.Boudourides@gmail.com

STOP